

Autonomous Job Application Agent

Problem Definition, Data Processing & Evaluation Criteria

1. Problem Definition

Tailoring a separate resume, cover letter, and interview-prep packet for every role is slow and repetitive. Off-the-shelf “AI resume writers” fail in two ways that matter for a real applicant: **(1) they fabricate** — inventing employers, titles, dates, metrics, or skills, which becomes a liability in an interview; and **(2) they leak personal data** — pasting a full resume into a third-party LLM exposes email, phone, address, and sometimes SSN / passport / credit-card numbers.

Problem statement

Build an autonomous, multi-agent system that, given a job description and a candidate’s resume, produces a tailored resume, cover letter, and interview-prep guide — without fabricating facts and without leaking sensitive personal data — while keeping a human in the loop for final approval.

Goals (priority order)

#	Goal	Why it matters
G1	No fabrication — every claim in generated docs traces back to the original resume	A false claim surfaces in interviews; trust is the core value
G2	PII protection — sensitive IDs never reach the LLM; standard PII redacted then restored in the final artifact	Privacy / regulatory exposure
G3	Relevance / JD alignment — reflect the role’s required skills, keywords, tone, seniority	The actual user value
G4	Human-in-the-loop — user reviews, gives per-document feedback, explicitly approves	User stays in control; the agent never auto-submits
G5	Quality — structured, concise, cliché-free, ATS-friendly documents	Determines competitiveness

Scope

In scope - Tailor an existing resume to one target JD (reframe, not rewrite) - Matched cover letter + interview-prep guide - Lightweight company research via public web search - PII detection, redaction, restoration - Human-feedback revision loop with per-document targeting - Export to Markdown / PDF	Out of scope (current version) - Auto-submitting to job boards / portals - Credential management or scraping behind logins - Authoring a resume from scratch - Multi-role batch tailoring in one run - Durable multi-user persistence (in-memory sessions)
---	---

Key constraints & assumptions

No fabrication: generation nodes may reframe/re-emphasize existing content but must not introduce new employers, titles, dates, metrics, or skills (prompt-enforced, verified in §3). **PII boundary:** analysis runs on a sanitized resume copy; raw PII is reattached only in the final artifact, and sensitive PII (SSN / credit-card / passport) aborts the run outright. **Assumptions:** the resume is truthful (the guarantee is relative to that input), and company research uses the free DuckDuckGo Instant Answer API (best-effort; a richer provider would swap in for production).

User interface

Users paste or upload the JD and resume, then review and approve the generated documents in the browser — a FastAPI web app (api.py, static/index.html) that streams node-by-node progress and collects per-document feedback.

2. Data Processing

Input	Req.	Formats	Handling
Job description	Yes	Text or PDF	PDF parsed to text; company name extracted by a cheap LLM call
Resume	Yes	Text / MD / PDF	Parsed via pypdf (layout-preserving); scanned/encrypted PDFs rejected; 10 MB cap

Processing pipeline (LangGraph state machine)

A 10-node LangGraph **StateGraph** (graph.py) over a shared **AgentState** TypedDict (state.py): parallel fan-out, conditional routing, an in-memory checkpointer, and an interrupt before human review.

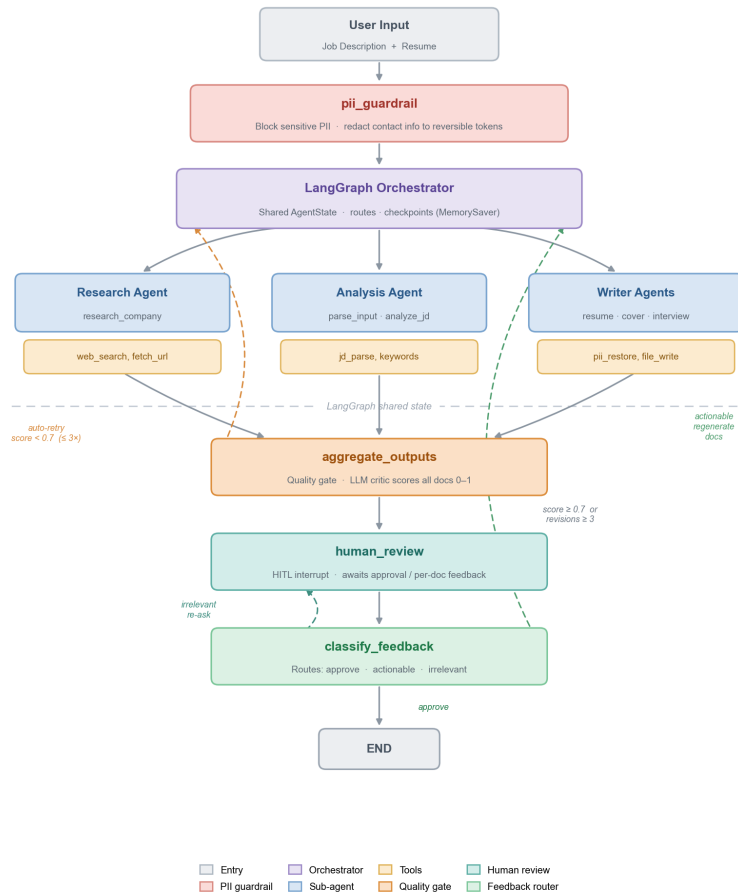


Figure 1 — PII guard, research / analysis / writer sub-agents, the quality gate, and the human-review feedback loops (auto-retry, regenerate, re-ask).

PII data layer

Tier	Examples (regex-detected)	Action
Sensitive	SSN, credit-card, passport number	Abort the run (PIIBlockedError) — never sent to the LLM
Standard	email, phone, LinkedIn/GitHub URL, generic URL, address	Redact → token → restore

Flow: scan resume → replace each match with a reversible token [PII:TYPE:uid] → store pii_mapping in state → analysis & generation nodes see only the sanitized resume → restore tokens when assembling the final document. `strip_stray_pii_tokens()` removes any hallucinated token; logs are one-way redacted. The LLM never receives raw contact details or government identifiers.

State model (key fields)

Group	Fields
Inputs	job_description, resume_text, company_name
PII layer	pii_report (counts), pii_mapping, sanitized_resume
Research / analysis	company_research, jd_analysis
Generated docs	tailored_resume, cover_letter, interview_questions (+ sanitized copies for revision context)
Quality	quality_score, quality_feedback, revision_count
Feedback	human_feedback, feedback_type, approved, resume_feedback, cover_letter_feedback, interview_feedback

Feedback semantics & outputs

classify_feedback resolves input by priority: (1) the web-UI Approve flag is the only way to approve all three docs at once → END; (2) per-document feedback is relevance-checked against its target — irrelevant text is discarded, actionable text regenerates *only* that document. **revision_count** increments only on actionable feedback (gibberish never burns a cycle), and each node revises its own previous draft.

Outputs: tailored resume, cover letter, interview prep, and company research are written as Markdown under `outputs/{company}_*.md` and exportable to PDF via `/api/export-pdf`. Sessions are held in memory by `session_id` (a production build would move to Redis / SQLite).

Tech stack: LangGraph (≥ 0.2) · a configurable chat model via LangChain · FastAPI + Uvicorn + single-page UI · pypdf / fpdf2 · DuckDuckGo Instant Answer · deepeval + pytest.

3. Evaluation Criteria

A runnable suite under `evals/` combines **LLM-as-judge** metrics (deepeval GEval + Faithfulness), **deterministic checks**, and **regression controls**. The two golden examples (`backend_engineer_stripe`, `data_scientist_airbnb`) use resumes deliberately narrower than the JD, so the no-fabrication metrics have a real chance to catch invented skills or seniority.

Metric	Type	What it checks	Pass $\geq$
No Fabrication	GEval	Tailored resume invents no employer / title / date / metric / skill	0.80
Cover Letter No Fabrication	GEval	Every cover-letter claim traces back to the resume	0.80
Interview Prep No Fabrication	GEval	STAR answers grounded in the resume; generic Q&A not penalized	0.80
Faithfulness	deepeval	Tailored-resume claims grounded in the source resume	0.80
JD Alignment	GEval	Surfaces JD-required skills the candidate genuinely has	0.70
Cover Letter Quality	GEval	Specific hook, real experience, no clichés, concise	0.70
No stray PII tokens	Deterministic	Final documents contain no leftover [PII:.*] tokens	0

Thresholds follow goal priority (\$1): **no-fabrication & faithfulness gate at 0.80** (a violation is a hard failure), **relevance & quality gate at 0.70**. **Regression controls** (`test_fabrication_regression.py`) guard the judges themselves: a fabricated doc must score below threshold, an honest doc must pass, and the fabricated version must score strictly below the faithful one — while generic technical Q&A is never flagged.

Pipeline, CI gate & runtime quality gate

evaluate.py generates outputs via the real API, caches them under `evals/fixtures/`, scores them, runs the PII-token leak check, and exits non-zero on any miss. **test_quality.py** is the CI gate — it scores cached fixtures only (judge model, no full re-run). At runtime, **aggregate_outputs** emits a `quality_score` in $[0, 1]$; **route_after_aggregate** requires ≥ 0.7 before the human, else loops back to regenerate, capped at `revision_count` ≥ 3 . Guide: 0.9+ publish-ready · 0.7–0.9 minor tweaks · < 0.7 rework.

```
# refresh fixtures (live agent run + judge)
python -m autonomous_job_application_agent.evals.evaluate --refresh
# CI gate – score cached fixtures only
pytest autonomous_job_application_agent/evals/test_quality.py
```

Success criteria

- ✓ No sensitive PII reached the LLM; final documents contain zero stray PII tokens.
- ✓ All no-fabrication / faithfulness metrics ≥ 0.80 ; JD-alignment & cover-letter quality ≥ 0.70 .
- ✓ Fabrication regression controls hold (fabricated $<$ threshold $<$ faithful).
- ✓ Runtime quality gate reached ≥ 0.7 (or hit the revision cap) and the human explicitly approved the artifacts.

Known limitations & future work

Search: DuckDuckGo Instant Answer is shallow (swap for Tavily / SerpAPI). **Persistence:** in-memory sessions are lost on restart (move to Redis / SQLite). **PII detection** is regex-based — an NER detector would raise recall. **Dataset:** two golden examples; broader role coverage would strengthen the signal. **No auto-submission** by design — the agent stops at approved artifacts.